DEVOPS · AI · ROADMAP

DevOps + AI Roadmap 2026

Lộ trình 12 tháng cho người mới và người chuyển hướng

Từ zero tới sẵn sàng đi làm - với kỹ năng AI thực chiến 2026

BIÊN SOẠN BỞI

Đinh Xuân Công

10 năm thực chiến · Dev / DevOps / Data Engineering

Talkshow: ***"DevOps Engineering - Tương lai Bứt phá trong ngành Software"***

Phiên bản 2026.01 · Tài liệu miễn phí - chỉ chia sẻ, không kinh doanh lại

MỤC LỤC

Lời mở đầu 3
Chương 1 - Bản đồ tổng quan 4
Chương 2 - Nền tảng: 10 bước cốt lõi 6
Bước 1 - Linux & CLI 6
Bước 2 - Networking cơ bản 7
Bước 3 - Git & Version Control 8
Bước 4 - Scripting (Python + Bash) 9
Bước 5 - CI/CD 10
Bước 6 - Cloud Platform (AWS ưu tiên) 11
Bước 7 - Container & Orchestration 12
Bước 8 - Infrastructure as Code (Terraform) 13
Bước 9 - Monitoring & Observability 14
Bước 10 - Security cơ bản (DevSecOps) 15
Chương 3 - Lớp kỹ năng AI 2026: 5 năng lực bứt phá 16
Chương 4 - Lộ trình theo 3 nhóm đối tượng 20
Chương 5 - Lộ trình 12 tháng (bản chung) 23
Chương 6 - Bốn kỹ năng AI không thay được 25
Chương 7 - Hành động 14 ngày đầu tiên 26
Chương 8 - 7 quy tắc vận hành cá nhân 27
Phụ lục A - Resource list (sách, khoá, kênh) 28
Phụ lục B - Glossary 30 thuật ngữ cốt lõi 29
Phụ lục C - Liên hệ & bước tiếp theo 30

LỜI MỞ ĐẦU

Tài liệu này dành cho ai và dùng thế nào

Tài liệu này không phải sách giáo khoa. Đây là lộ trình thực chiến em đã rút ra sau 8 năm làm Dev, DevOps và Data Engineering - vừa làm production thật, vừa đào tạo cho hàng nghìn lập trình viên. Tài liệu bám sát nội dung buổi talkshow và đi sâu hơn ở phần lộ trình - thứ không kịp nói chi tiết trong 70 phút.

Đối tượng chính: sinh viên IT năm 2-4, fresher/junior developer muốn chuyển hướng sang DevOps, và những người đang làm IT (System Admin, QA, Tester) muốn nhảy sang nghề DevOps/SRE.

Lộ trình thiết kế cho 12 tháng - không phải vì đây là con số đẹp, mà vì đó là khoảng thời gian thực tế để một người mới đi từ zero đến trình độ sẵn sàng đi làm, NẾU học đúng thứ tự và có dự án portfolio thật. Với người đã có nền code, thời gian rút ngắn còn 6-9 tháng.

> **TRIẾT LÝ CỦA TÀI LIỆU** *Học công cụ thì dễ - học tư duy thì khó. Tài liệu này ưu tiên TƯ DUY HỆ THỐNG và NGUYÊN LÝ trước, công cụ sau.*

> *Mỗi bước có 1 mini-project bắt buộc, đẩy lên GitHub. Không có portfolio, không có khả năng đi làm. Đơn giản vậy.*

> *AI không xoá nghề. AI xoá phần việc lặp lại trong nghề. Cả lộ trình được thiết kế để bạn ở phía 'điều khiển AI', không phải 'bị AI thay'.*

Bốn nguyên tắc dùng tài liệu hiệu quả

- **Đi theo thứ tự.** 10 bước nền tảng được sắp xếp có chủ đích. Học CI/CD trước khi nắm Git - bạn sẽ thấy quá tải. Học K8s trước khi hiểu Docker - bạn sẽ học vẹt. Đừng nhảy cóc.
- **Không học để biết - học để làm.** Mỗi bước phải có mini-project commit lên GitHub. Không có dự án, không tính là đã học.
- **Một công cụ, học sâu trước.** Đừng học cả Terraform, Pulumi, Crossplane cùng lúc. Học sâu Terraform trước. Cái khác là biến thể của cùng tư duy.
- **Mỗi tuần review một lần.** Cuối tuần, mở checklist (file Skills_Checklist_2026.xlsx kèm theo), tự đánh giá. Nhìn lại 4 tuần - cảm giác tiến bộ là động lực lớn nhất.

> **LƯU Ý QUAN TRỌNG** *Tài liệu này KHÔNG thay thế trải nghiệm thực chiến. Nó là bản đồ - không phải chuyến đi.*

> *Nếu sau 3 tháng đầu bạn chưa commit gì lên GitHub, hãy dừng lại và nghĩ xem mình đang HỌC hay đang TIÊU THỤ NỘI DUNG.*

> *Mọi con số trong tài liệu (giờ học, thời gian, tỉ lệ) là ước lượng dựa trên kinh nghiệm đào tạo của tác giả. Tốc độ thật phụ thuộc nền tảng cá nhân và thời gian dành ra mỗi ngày.*

CHƯƠNG 1 - Bản đồ tổng quan

Hai lớp năng lực, một con đường

DevOps Engineer 2026 cần hai lớp năng lực rõ ràng. Thiếu lớp nào cũng không đủ.

Lớp 1 - Nền tảng (điều kiện CẦN)

Đây là phần kiến thức không thể tắt: Linux, Networking, Git, Scripting, CI/CD, Cloud, Container, IaC, Monitoring, Security. Mười bước - không có đường tắt. AI không thay được lớp này, AI chỉ giúp anh học và vận hành nhanh hơn. Người không có nền tảng vững có thể tự dùng được vài công cụ, nhưng không bao giờ làm được DevOps Engineer đúng nghĩa.

Lớp 2 - Kỹ năng AI 2026 (điều kiện ĐỦ)

Đây là lớp mới - chỉ vừa hình thành 2 năm trở lại đây: AIOps, Platform Engineering & IDP, MCP & AI Agent Infrastructure, FinOps, Security-as-Code. Lớp này phân biệt người 'làm DevOps cũ' với người 'DevOps 2026'. Người có cả hai lớp = người ở cột CƠ, lương thường cao hơn DevOps thuần khoảng 27% theo các báo cáo thị trường 2026.

> **NGUYÊN LÝ KÉP** *Nền tảng giúp bạn ĐƯỢC TUYỂN. Lớp AI 2026 giúp bạn BỨT PHÁ và khó bị thay thế.*

> *Không có lớp 1, lớp 2 không bám được - học MCP mà không biết Kubernetes thì chỉ là gọi API.*

Bốn cột mốc trên con đường

Lộ trình 12 tháng được chia thành 4 cột mốc rõ ràng. Mỗi cột mốc có tiêu chí định lượng - không dựa vào cảm tính "mình thấy ổn".

Cột mốc	Thời gian	Trạng thái sau khi đạt	Tiêu chí định lượng
1. HỌC	Tháng 1-3	Nắm vững Linux, Git, Networking, Scripting	3 mini-project trên GitHub; trả lời được 80% câu hỏi cơ bản về OS/networking
2. LÀM	Tháng 4-7	Triển khai được app end-to-end với CI/CD, Cloud, Container, IaC	1 dự án full-stack có pipeline tự động deploy lên cloud thật
3. BỨT PHÁ	Tháng 8-10	Có Monitoring, Security, và bắt đầu lớp AI 2026	Hệ thống có metrics + log + alert; quét security tự động trong pipeline
4. ĐỊNH VỊ	Tháng 11-12	Sẵn sàng đi làm hoặc lên Mid-level	CV + LinkedIn + portfolio sẵn sàng; pass được phỏng vấn 1 vòng technical

Phân bổ thời gian khuyến nghị

12 tháng = ~52 tuần. Nếu mỗi tuần dành 15 giờ học có chủ đích (~2 giờ/ngày + cuối tuần), tổng là ~780 giờ. Đây là con số khả thi cho người đang đi học hoặc đi làm bán thời gian.

Phân bổ giờ tham khảo:

- **Lý thuyết & video:** 30% - đủ để hiểu khái niệm, không sa đà.
- **Làm dự án thực hành (hands-on):** 50% - đây là nơi kỹ năng thật sự hình thành.

- **Đọc tài liệu chính thức / blog kỹ thuật:** 15% - sách + official docs, không phải tweet và TikTok.
- **Cộng đồng & mentor:** 5% - tham gia 1-2 community, không cần nhiều.

CHƯƠNG 2 - Nền tảng: 10 bước cốt lõi

Mười bước dưới đây là phần "điều kiện cần". Phải đi theo đúng thứ tự. Mỗi bước có cấu trúc thống nhất:

- **Mục tiêu -** bạn sẽ làm được gì sau khi học xong.
- **Concept cốt lõi -** không học cái này thì không gọi là "biết".
- **Công cụ chính -** ưu tiên 1 công cụ chuẩn, không học dàn trải.
- **Mini-project -** dự án nhỏ commit lên GitHub. Bắt buộc.
- **Thời gian dự kiến -** tham khảo, tuỳ nền tảng cá nhân.
- **Tiêu chí "xong" -** checklist nội bộ để biết khi nào đủ điều kiện sang bước kế.
- **Resource -** 1 sách chính + 1 khoá học + 1 doc chính thức.

01 Linux & CLI

Thời gian đề xuất: 3-4 tuần · ~50 giờ

MỤC TIÊU

Thao tác hệ điều hành Linux qua dòng lệnh thành thạo, không cần GUI. Đây là môi trường server thực tế của 99% production workload.

CONCEPT CỐT LÕI CẦN NẮM

- File system: cấu trúc /etc, /var, /usr, /home, /opt - tại sao chia như vậy
- Process management: ps, top, kill, nohup, signals (SIGTERM vs SIGKILL)
- Permissions: chmod, chown, sticky bit, ACL, sudo và cách dùng đúng
- Networking commands: ip, ss, netstat, ping, traceroute, dig, curl
- Service management: systemd, journalctl, service unit files
- Text processing: grep, sed, awk, cut, sort, uniq, xargs

CÔNG CỤ CHÍNH

Ubuntu 22.04 LTS hoặc Debian 12 (trên WSL2 cho Windows hoặc VirtualBox/UTM cho Mac). Editor: vim (học vim căn bản 30 lệnh).

MINI-PROJECT BẮT BUỘC

> **ĐẨY LÊN GITHUB KHI XONG** *Viết shell script tự động hoá 3 task vận hành: (1) backup folder /etc lên S3 hằng đêm, (2) monitor disk usage và gửi alert qua webhook khi >80%, (3) archive log >30 ngày sang folder cold. Push lên GitHub, README có hướng dẫn cài và demo.*

TIÊU CHÍ "XONG" - CHECKLIST TỰ ĐÁNH GIÁ

- ☐ Cài được Ubuntu, ssh vào server, dùng được tmux để giữ session
- ☐ Hiểu sự khác biệt giữa quá trình foreground/background, demo được
- ☐ Viết được script bash 50+ dòng, có error handling và logging
- ☐ Đọc được file unit của systemd, tạo service tự khởi động được

RESOURCE CỐT LÕI

Sách: "The Linux Command Line" - William Shotts (PDF miễn phí trên LinuxCommand.org)

Khoá học: Linux Journey (https://linuxjourney.com) - free, học interactive

Docs / Reference: man pages của các lệnh trên - hãy đọc man chứ đừng google snippet

02 Networking cơ bản

Thời gian đề xuất: 2-3 tuần · ~30 giờ

MỤC TIÊU

Hiểu mạng máy tính ở mức đủ để debug khi production có vấn đề: tại sao service A không gọi được service B, tại sao DNS chậm, tại sao certificate không trust.

CONCEPT CỐT LÕI CẦN NẮM

- Mô hình OSI 7 lớp và TCP/IP 4 lớp - biết mỗi vấn đề thuộc lớp nào
- DNS: A, AAAA, CNAME, MX, TXT, TTL; recursive vs authoritative resolver
- HTTP/HTTPS: methods, status codes, headers, keepalive, HTTP/2 vs HTTP/3
- TLS handshake: certificate chain, SNI, mutual TLS
- Load balancer: L4 vs L7, sticky session, health check
- NAT, firewall, security group, network ACL - nguyên lý hoạt động

CÔNG CỤ CHÍNH

dig, nslookup, curl -v, openssl s_client, tcpdump (chỉ đọc kết quả, không cần master), Wireshark cho phần học handshake.

MINI-PROJECT BẮT BUỘC

> **ĐẨY LÊN GITHUB KHI XONG** *Setup home lab với 2 VM (Ubuntu) trên cùng 1 network. Cấu hình DNS server (BIND9 hoặc dnsmasq) trên VM1, resolve được tên service trên VM2. Capture TLS handshake giữa client và 1 HTTPS site bằng tcpdump, phân tích kết quả trong README.*

TIÊU CHÍ "XONG" - CHECKLIST TỰ ĐÁNH GIÁ

- ☐ Vẽ được sơ đồ mô tả 1 request HTTPS đi từ browser đến server
- ☐ Giải thích được TLS handshake từng bước, kèm output thật của openssl
- ☐ Debug được lỗi DNS bằng dig, biết khác giữa A và CNAME khi nào dùng cái nào
- ☐ Biết khác giữa load balancer L4 và L7, ví dụ use case cụ thể

RESOURCE CỐT LÕI

Sách: "Computer Networking: A Top-Down Approach" - Kurose & Ross (chỉ đọc chương 1-6)

Khoá học: NetworkChuck trên YouTube - playlist “CCNA”, free và rất dễ hiểu

Docs / Reference: Cloudflare Learning Center - https://www.cloudflare.com/learning/ - chất lượng best in class

03 Git & Version Control

Thời gian đề xuất: 1-2 tuần · ~20 giờ

MỤC TIÊU

Dùng Git thành thạo trong workflow nhóm, hiểu sâu hơn “add → commit → push”. Đây là kỹ năng nền cho mọi thứ phía sau, không chỉ DevOps.

CONCEPT CỐT LÕI CẦN NẮM

- Snapshot vs delta - Git lưu gì khi commit, tại sao branch rẻ
- Branching strategies: GitFlow, GitHub Flow, trunk-based - chọn cái nào và khi nào
- Merge vs rebase - khác biệt, khi nào dùng cái nào, conflict resolution
- git reset (soft/mixed/hard), revert, cherry-pick - phân biệt và biết khi nào dùng
- Git internals: object database (blob/tree/commit/tag), HEAD, reflog
- Hooks: pre-commit, pre-push - tự động hoá quality gate cục bộ

CÔNG CỤ CHÍNH

git CLI (không dùng GUI quá nhiều trong giai đoạn học), GitHub, pre-commit framework.

MINI-PROJECT BẮT BUỘC

> **ĐẨY LÊN GITHUB KHI XONG** *Tạo 1 repo cá nhân theo trunk-based development. Setup pre-commit hooks (lint code + format). Mở PR ngược lại 1 open-source project (sửa typo cũng được - quan trọng là quy trình). Commit message theo Conventional Commits.*

TIÊU CHÍ "XONG" - CHECKLIST TỰ ĐÁNH GIÁ

- ☐ Giải thích được rebase khác merge bằng vẽ Git graph
- ☐ Resolve được merge conflict không cần Google
- ☐ Đọc được reflog và phục hồi được commit “lỡ tay xoá”
- ☐ Có ít nhất 1 PR được merge vào open-source project ngoài

RESOURCE CỐT LÕI

Sách: "Pro Git" - Scott Chacon & Ben Straub (free online tại git-scm.com)

Khoá học: Atlassian Git Tutorial - https://www.atlassian.com/git/tutorials

Docs / Reference: git help <command> - đọc kỹ man trước khi tra StackOverflow

04 Scripting (Python + Bash)

Thời gian đề xuất: 3-4 tuần · ~50 giờ

MỤC TIÊU

Tự động hoá tác vụ vận hành. Bash cho task nhỏ trong shell, Python cho logic phức tạp, gọi API, xử lý JSON/YAML.

CONCEPT CỐT LÕI CẦN NẮM

- Bash: argv, exit codes, redirection, pipe, here-doc, trap, set -euo pipefail
- Python: virtualenv/venv, requirements.txt vs poetry, argparse, logging
- Regex: cơ bản (ký tự, group, quantifier) - đủ để dùng grep/sed/Python re
- Parsing: JSON (jq + Python), YAML (PyYAML), TOML
- HTTP client: curl trong bash, requests/httpx trong Python
- Error handling: exit codes, try/except, retry với exponential backoff

CÔNG CỤ CHÍNH

Python 3.10+, bash, jq, httpx hoặc requests, click hoặc argparse.

MINI-PROJECT BẮT BUỘC

> **ĐẨY LÊN GITHUB KHI XONG** *Viết Python CLI tool gọi GitHub API, lấy danh sách PR đang mở của 1 repo, output report Markdown. Có flag --since=, --label=. Đóng gói thành package cài được bằng pip install ./. Cấu hình ruff/black để format.*

TIÊU CHÍ "XONG" - CHECKLIST TỰ ĐÁNH GIÁ

- ☐ Viết được bash script 100+ dòng có error handling tử tế (set -euo pipefail, trap)
- ☐ Viết được Python CLI tool có argparse, logging, retry logic
- ☐ Đọc được regex phức tạp, viết được regex đơn giản không Google
- ☐ Đóng gói được Python package và chạy được trên máy khác qua pip

RESOURCE CỐT LÕI

Sách: "Automate the Boring Stuff with Python" - Al Sweigart (free online tại automatetheboringstuff.com)

Khoá học: CS50P - Harvard "Introduction to Programming with Python" (free trên edX)

Docs / Reference: Python docs phần “Tutorial” + “Library Reference” cho modules os/sys/subprocess/json

05 CI/CD

Thời gian đề xuất: 2-3 tuần · ~35 giờ

MỤC TIÊU

Tự động hoá vòng đời phần mềm: từ commit → test → build → deploy. Đây là DNA của DevOps, không có không tính.

CONCEPT CỐT LÕI CẦN NẮM

- Pipeline: stages, jobs, steps, artifacts; phân biệt CI vs CD vs CDe (Continuous Delivery vs Deployment)
- Quality gates: lint, unit test, integration test, coverage threshold
- Build artifacts: container image, npm package, binary - strategies cache build
- Secrets management: GitHub Actions Secrets, environment-scoped, OIDC
- Matrix builds: chạy song song trên nhiều OS / version
- Deployment strategies: rolling, blue-green, canary

CÔNG CỤ CHÍNH

GitHub Actions là mặc định (~70% thị trường). Biết GitLab CI là +. Argo CD là kiến thức bonus.

MINI-PROJECT BẮT BUỘC

> **ĐẨY LÊN GITHUB KHI XONG** *Setup pipeline đầy đủ cho project ở bước 4: (a) lint với ruff, (b) test với pytest, (c) build Python package, (d) publish lên TestPyPI khi tag git được tạo, (e) deploy docs lên GitHub Pages. Có matrix build trên Python 3.10/3.11/3.12.*

TIÊU CHÍ "XONG" - CHECKLIST TỰ ĐÁNH GIÁ

- ☐ Setup được pipeline 4 stage trở lên, có cache artifact giữa các stage
- ☐ Phân biệt được khi nào dùng env vs secret vs variable
- ☐ Biết debug khi pipeline đỏ - đọc log có hệ thống, không đoán mò
- ☐ Hiểu được deployment strategies, vẽ được sơ đồ rolling vs blue-green

RESOURCE CỐT LÕI

Sách: "Continuous Delivery" - Jez Humble & David Farley (sách kinh điển)

Khoá học: GitHub Actions Learning Lab + Lab “DevOps with GitHub” trên Microsoft Learn (free)

Docs / Reference: https://docs.github.com/en/actions - đọc kỹ phần Workflow syntax

06 Cloud Platform (AWS ưu tiên)

Thời gian đề xuất: 4-5 tuần · ~70 giờ

MỤC TIÊU

Hiểu hạ tầng cloud ở mức triển khai và vận hành 1 hệ thống thật. AWS là lựa chọn pragmatic vì thị phần và độ phổ biến trong JD tuyển dụng VN.

CONCEPT CỐT LÕI CẦN NẮM

- IAM: user, group, role, policy, trust relationship - least privilege
- VPC: subnet (public vs private), route table, NAT gateway, security group vs NACL
- Compute: EC2 (instance types, AMI), Lambda, ECS/Fargate
- Storage: S3 (bucket policy, versioning, lifecycle), EBS (volume types), EFS
- Database: RDS (Aurora, Multi-AZ), DynamoDB
- Networking & DNS: Route 53, Application Load Balancer, CloudFront
- Logging & cost: CloudWatch Logs, AWS Cost Explorer, Budget Alert

CÔNG CỤ CHÍNH

AWS CLI (chính), AWS Console (đôi khi), AWS CloudShell. Có thể dùng LocalStack để học miễn phí phần lớn.

MINI-PROJECT BẮT BUỘC

> **ĐẨY LÊN GITHUB KHI XONG** *Deploy 1 web app 3-tier trên AWS: (a) frontend tĩnh trên S3 + CloudFront, (b) backend trên EC2 hoặc ECS Fargate, (c) RDS MySQL/Postgres ở private subnet. Document architecture diagram trong README. Setup CloudWatch alarm cho CPU > 80%.*

TIÊU CHÍ "XONG" - CHECKLIST TỰ ĐÁNH GIÁ

- ☐ Hiểu IAM đủ sâu để viết policy với least-privilege, không dùng AdministratorAccess
- ☐ Vẽ được sơ đồ VPC với public/private subnet, route table, NAT gateway
- ☐ Biết khi nào dùng EC2 vs Lambda vs Fargate - trade-off, không phải “chọn cái mới”
- ☐ Đọc được hoá đơn AWS, biết chi phí lớn nhất nằm ở dịch vụ nào

RESOURCE CỐT LÕI

Sách: "AWS Certified Solutions Architect Associate Study Guide" - Ben Piper (chỉ đọc, không cần thi)

Khoá học: Stephane Maarek - AWS Certified Solutions Architect Associate (Udemy, giảm giá thường xuyên về ~$15)

Docs / Reference: AWS Skill Builder - https://skillbuilder.aws - free tier có nhiều lab tốt

07 Container & Orchestration

Thời gian đề xuất: 4-5 tuần · ~65 giờ

MỤC TIÊU

Đóng gói và điều phối ứng dụng ở quy mô lớn. Đây là nền của hầu hết workload production hiện đại.

CONCEPT CỐT LÕI CẦN NẮM

- Container vs VM: namespace, cgroup, image layer
- Dockerfile best practices: multi-stage build, layer caching, .dockerignore, distroless image
- Docker networking: bridge, host, overlay; phân biệt EXPOSE và publish
- Kubernetes core: Pod, Deployment, Service, Ingress, ConfigMap, Secret, PVC
- Workload types: Deployment vs StatefulSet vs DaemonSet vs Job/CronJob
- Resource management: requests vs limits, QoS class, HPA
- Helm chart: template, values, release

CÔNG CỤ CHÍNH

Docker Desktop hoặc Podman. Kubernetes: kind hoặc minikube cho local; sau đó đụng EKS thật. Helm 3.

MINI-PROJECT BẮT BUỘC

> **ĐẨY LÊN GITHUB KHI XONG** *Containerize app ở bước 4: viết Dockerfile multi-stage (build → runtime), tối ưu để image < 100MB. Tạo Helm chart deploy lên local K8s (kind). Có Service + Ingress, ConfigMap cho config, Secret cho API key. Setup HPA scale theo CPU.*

TIÊU CHÍ "XONG" - CHECKLIST TỰ ĐÁNH GIÁ

- ☐ Viết được Dockerfile multi-stage, hiểu vì sao tách build stage và runtime stage
- ☐ Debug được pod CrashLoopBackOff bằng kubectl describe + logs
- ☐ Hiểu sự khác biệt giữa Deployment và StatefulSet, ví dụ cụ thể
- ☐ Viết được Helm chart đơn giản (template, values.yaml, helpers)

RESOURCE CỐT LÕI

Sách: "Kubernetes Up & Running" - Brendan Burns, Joe Beda (O'Reilly)

Khoá học: Nigel Poulton - "Getting Started with Kubernetes" trên Pluralsight; KodeKloud cũng tốt và rẻ

Docs / Reference: https://kubernetes.io/docs/concepts/ - đọc lần lượt, đừng nhảy

08 Infrastructure as Code (Terraform)

Thời gian đề xuất: 2-3 tuần · ~35 giờ

MỤC TIÊU

Khai báo hạ tầng bằng code, version control được, có thể recreate môi trường ở bất cứ đâu trong 5 phút.

CONCEPT CỐT LÕI CẦN NẮM

- Declarative vs imperative - tại sao Terraform khác Ansible
- Provider, resource, data source
- State management: state file, remote state (S3 + DynamoDB lock), workspace
- Module: input variables, output, module composition
- Lifecycle: create_before_destroy, prevent_destroy, ignore_changes
- Drift detection: terraform plan để phát hiện thay đổi ngoài code

CÔNG CỤ CHÍNH

Terraform 1.6+ (hoặc OpenTofu nếu lo về license), AWS provider, tflint, tfsec.

MINI-PROJECT BẮT BUỘC

> **ĐẨY LÊN GITHUB KHI XONG** *Recreate toàn bộ AWS infrastructure ở bước 6 bằng Terraform - không click console nào. Tách thành module (vpc, compute, database). Lưu state trên S3 với DynamoDB lock. Setup tflint + tfsec trong pipeline ở bước 5.*

TIÊU CHÍ "XONG" - CHECKLIST TỰ ĐÁNH GIÁ

- ☐ Viết được Terraform module reuse được, có input/output rõ ràng
- ☐ Setup được remote state với S3 + DynamoDB, hiểu vì sao cần locking
- ☐ Hiểu sự khác biệt giữa terraform apply và terraform import, khi nào dùng
- ☐ Đọc được state file (không sửa tay) và biết cách rescue khi state lệch

RESOURCE CỐT LÕI

Sách: "Terraform: Up & Running" - Yevgeniy Brikman (O'Reilly)

Khoá học: HashiCorp Learn - https://developer.hashicorp.com/terraform/tutorials (free, chính chủ)

Docs / Reference: https://developer.hashicorp.com/terraform/docs

09 Monitoring & Observability

Thời gian đề xuất: 2-3 tuần · ~35 giờ

MỤC TIÊU

Biết được hệ thống đang chạy thế nào trước khi nó hỏng. Đây là kỹ năng phân biệt người “làm xong” và người “làm chín”.

CONCEPT CỐT LÕI CẦN NẮM

- 3 trụ cột observability: Metrics, Logs, Traces - khác biệt và khi nào dùng cái nào
- SLI / SLO / SLA - định nghĩa, ví dụ, error budget
- Alert design: actionable, không alert noise; ngưỡng tĩnh vs động
- Cardinality: tại sao đừng cho user_id vào label của metric
- OpenTelemetry: instrumentation tự động vs manual
- Log aggregation: structured logging, log levels, correlation ID

CÔNG CỤ CHÍNH

Prometheus (metrics), Grafana (visualize), Loki hoặc ELK (logs), Jaeger hoặc Tempo (traces). OpenTelemetry SDK.

MINI-PROJECT BẮT BUỘC

> **ĐẨY LÊN GITHUB KHI XONG** *Deploy Prometheus + Grafana + Loki trên K8s cluster ở bước 7. Instrument app Python với OpenTelemetry, expose /metrics endpoint. Tạo Grafana dashboard với 4 panel cốt lõi (RED method: Rate, Errors, Duration; cộng saturation). Viết 3 alert rule có ý nghĩa.*

TIÊU CHÍ "XONG" - CHECKLIST TỰ ĐÁNH GIÁ

- ☐ Viết được PromQL truy vấn cơ bản (rate, histogram_quantile, sum by)
- ☐ Phân biệt được khi nào dùng counter vs gauge vs histogram vs summary
- ☐ Định nghĩa được SLO cho app của mình và tính error budget
- ☐ Biết alert nào là actionable, alert nào nên xoá

RESOURCE CỐT LÕI

Sách: "Site Reliability Engineering" - Google (free online tại sre.google/books)

Khoá học: Prometheus & Grafana Fundamentals - https://prometheus.io/docs/tutorials/getting_started/

Docs / Reference: https://opentelemetry.io/docs/ - phần "Concepts" trước

10 Security cơ bản (DevSecOps)

Thời gian đề xuất: 2-3 tuần · ~35 giờ

MỤC TIÊU

Tích hợp security vào CI/CD, không phải dán thêm cuối quy trình. Đây là kỹ năng đắt giá vì ít DevOps junior nắm được.

CONCEPT CỐT LÕI CẦN NẮM

- Shift-left security: scan ở từng giai đoạn, fail fast
- Vulnerability scanning: SAST, DAST, SCA - khác biệt
- Container security: scan image, distroless, không chạy root
- Secrets management: không bao giờ commit secret, vault patterns
- IaC security: scan Terraform/K8s manifest trước apply
- Supply chain: SBOM, signed image (cosign), provenance

CÔNG CỤ CHÍNH

trivy hoặc grype (scan image + filesystem), gitleaks (secrets), checkov hoặc tfsec (IaC), HashiCorp Vault hoặc AWS Secrets Manager.

MINI-PROJECT BẮT BUỘC

> **ĐẨY LÊN GITHUB KHI XONG** *Bổ sung 4 security gate vào pipeline ở bước 5: (a) gitleaks scan repo cho leaked secret, (b) trivy scan dependency, (c) trivy scan Docker image sau khi build, (d) checkov scan Terraform code. Mỗi gate fail thì pipeline đỏ.*

TIÊU CHÍ "XONG" - CHECKLIST TỰ ĐÁNH GIÁ

- ☐ Viết được pipeline có 4 security gate, hiểu rõ mỗi cái làm gì
- ☐ Biết khác giữa CVE và CWE, biết đọc CVSS score
- ☐ Không bao giờ commit secret - dùng vault hoặc secrets manager cho mọi credential
- ☐ Hiểu khái niệm SBOM, sign image, supply chain attack

RESOURCE CỐT LÕI

Sách: "Container Security" - Liz Rice (O'Reilly)

Khoá học: OWASP DevSecOps Guideline - https://owasp.org/www-project-devsecops-guideline/

Docs / Reference: https://www.aquasec.com/cloud-native-academy/ - academy của Trivy, miễn phí, chất lượng cao

CHƯƠNG 3 - Lớp kỹ năng AI 2026: 5 năng lực bứt phá

Nếu nền tảng là điều kiện CẦN, lớp dưới đây là điều kiện ĐỦ để bứt phá. Năm năng lực này phân biệt người DevOps 2026 với người DevOps cũ. Theo các báo cáo thị trường, Platform Engineer kiếm cao hơn DevOps thuần khoảng 27%, và quy mô thị trường AIOps dự báo đạt ~$36.6 tỷ vào 2030. Đây là cửa sổ cơ hội đang mở rộng - nhưng có thời hạn.

> **KHI NÀO BẮT ĐẦU LỚP NÀY?** *Sau khi hoàn thành 7/10 bước nền tảng. Đừng vội. Học MCP mà không biết Kubernetes thì chỉ là gọi API.*

> *Không cần học cả 5. Chọn 2 cái khớp với hướng nghề bạn muốn - đào sâu, làm dự án. Mỗi năng lực dưới đây có thể nuôi sống cả một sự nghiệp.*

1. AIOps & Observability nâng cao

TẠI SAO QUAN TRỌNG

Alert fatigue đang ảnh hưởng khoảng 73% doanh nghiệp. Quá nhiều alert → on-call kiệt sức → bỏ alert thật. AIOps áp dụng ML/AI để giảm noise, tự động phân loại, và xử lý phần lớn alert trước khi đến tay con người.

CÔNG CỤ CHÍNH

- OpenTelemetry - chuẩn instrumentation đa ngôn ngữ, là nền cho mọi AIOps tooling
- Datadog AIOps, Dynatrace Davis AI - solution thương mại trưởng thành
- PromQL anomaly detection (z-score, EWMA, holt-winters) - bắt đầu từ đây vì free
- OpenAI API hoặc Claude API - tích hợp LLM vào alert triage flow

CÁCH BẮT ĐẦU

Mở rộng project bước 9 (Monitoring) - thêm Python script chạy mỗi 5 phút, lấy 100 alert gần nhất từ Alertmanager, gọi LLM để phân loại (severity, root cause hint), output webhook vào Slack thay vì email đại trà.

DỰ ÁN PORTFOLIO

> **ĐẨY LÊN GITHUB** *Self-healing CI pipeline - pipeline đỏ → agent đọc log → phân tích → propose patch → apply → re-run test → xanh. Đây chính là demo trong buổi talkshow.*

2. Platform Engineering & IDP (Internal Developer Platform)

TẠI SAO QUAN TRỌNG

Gartner dự báo ~80% tổ chức software sẽ có Platform team vào 2026, tăng từ ~55% năm 2025. Lương Platform Engineer cao hơn DevOps thuần khoảng 27%. Đây là kỹ năng nhu cầu vượt cung mạnh nhất trong 5 năm tới.

CÔNG CỤ CHÍNH

- Backstage (Spotify) - open source, là chuẩn de-facto
- Port - IDP-as-a-service, dễ bắt đầu hơn Backstage
- Crossplane - quản lý hạ tầng đa cloud bằng K8s API

- Kratix - framework để tự xây IDP từ scratch

CÁCH BẮT ĐẦU

Clone Backstage, deploy local, build 1 service template (Node.js + GitHub Actions + Dockerfile). Khi developer chạy backstage create-component, template tự sinh ra repo, CI pipeline, K8s manifest. Đây là 'golden path' tối giản nhất.

DỰ ÁN PORTFOLIO

> **ĐẨY LÊN GITHUB** *IDP prototype cho team 5 người: portal self-service tạo môi trường dev, kèm scaffolding code (3 service template), tích hợp với Argo CD để deploy tự động.*

3. MCP & AI Agent Infrastructure

TẠI SAO QUAN TRỌNG

MCP (Model Context Protocol) - chuẩn kết nối agent AI với hệ thống thật - đang nổi lên 2025-2026. Vị trí 'AI Platform Engineer' / 'Agent Infra Engineer' cực mới, ít người cạnh tranh. Người đi sớm có lợi thế lớn - 2 năm nữa sẽ đông, bây giờ thì chưa.

CÔNG CỤ CHÍNH

- MCP Python SDK / TypeScript SDK - chính thức từ Anthropic
- Claude Agent SDK, OpenAI Function Calling, Gemini Function Calling
- Trong agentic workflow: LangGraph, AutoGen, CrewAI (lựa chọn framework)
- Sandboxing cho agent: Firecracker, gVisor, Docker với seccomp

CÁCH BẮT ĐẦU

Đọc MCP spec (https://modelcontextprotocol.io). Viết 1 MCP server đơn giản: expose 3 tool (list_pods, get_logs, restart_pod) cho K8s cluster. Kết nối với Claude Desktop hoặc agent CLI.

DỰ ÁN PORTFOLIO

> **ĐẨY LÊN GITHUB** *DevOps Agent multi-tool: MCP server expose Kubernetes + Prometheus + GitHub. Agent có thể trả lời các câu như 'tìm service nào đang có error rate > 5% trong 1 giờ qua và rollback về version trước'. Đây là demo cấp expert.*

4. FinOps

TẠI SAO QUAN TRỌNG

Cloud cost runaway là vấn đề phổ biến. FinOps trở thành nghề riêng, có chứng chỉ chính thức (FinOps Foundation). DevOps có kỹ năng FinOps thường được trao quyền quyết định kiến trúc - đó là vị trí có ảnh hưởng cao.

CÔNG CỤ CHÍNH

- AWS Cost Explorer, AWS Budgets - cơ bản, phải biết
- Kubecost / OpenCost - cost allocation cho Kubernetes
- Infracost - ước tính cost từ Terraform code, đặt gate trong PR
- CloudHealth, Apptio Cloudability - solution thương mại

CÁCH BẮT ĐẦU

Setup tag policy đầy đủ cho AWS account (Environment, Service, Owner, CostCenter). Tạo CloudWatch dashboard hiển thị chi phí theo tag. Cài Infracost trong pipeline, comment cost diff vào PR mỗi lần thay đổi Terraform.

DỰ ÁN PORTFOLIO

> **ĐẨY LÊN GITHUB** *FinOps dashboard cho 1 AWS account: hiển thị chi phí theo service, theo team, theo môi trường. Có anomaly detection (chi phí spike > 30% so với tuần trước). Generate báo cáo tháng tự động.*

5. Security-as-Code & Policy-as-Code

TẠI SAO QUAN TRỌNG

Tốc độ AI sinh code → cần guardrail tự động. Security-as-Code đảm bảo policy được áp dụng nhất quán, không phụ thuộc vào việc developer 'có nhớ check không'.

CÔNG CỤ CHÍNH

- OPA (Open Policy Agent) + Gatekeeper - policy cho Kubernetes
- Kyverno - alternative dễ học hơn OPA cho K8s
- Checkov, tfsec - policy cho Terraform
- Cosign + Sigstore - sign image, supply chain security

CÁCH BẮT ĐẦU

Viết 5 OPA policy ngăn antipattern phổ biến trong K8s: (1) container chạy root, (2) image dùng tag latest, (3) thiếu resource limits, (4) thiếu liveness/readiness probe, (5) Service expose qua NodePort. Apply qua Gatekeeper.

DỰ ÁN PORTFOLIO

> **ĐẨY LÊN GITHUB** *Policy-as-code pipeline đầy đủ: 15+ rule cho Terraform + Kubernetes + Docker. Gate trong CI/CD: phải pass mọi policy mới deploy được. Generate compliance report HTML mỗi tuần.*

CHƯƠNG 4 - Lộ trình theo 3 nhóm đối tượng

Cùng một đích đến, ba điểm xuất phát khác nhau. Tài liệu chia ra 3 nhóm - bạn nên đọc phần của nhóm mình kỹ, lướt qua 2 nhóm còn lại để hiểu bối cảnh.

Nhóm	Xuất phát	Lợi thế tận dụng	Đích đến	Thời gian
A. Sinh viên IT	Năm 2-4, đã có nền lập trình cơ bản	Thời gian, môi trường học, sự tò mò	Internship hoặc Junior DevOps	12 tháng (song song học)
B. Fresher / Junior Dev	0-2 năm code, biết Git và 1 ngôn ngữ	Skill code, hiểu phần mềm	DevOps Engineer / SRE	6-9 tháng tập trung
C. IT chuyển hướng	Đang làm Ops / QA / Tester	Hiểu hệ thống, kinh nghiệm vận hành	DevOps Engineer / Platform	6-9 tháng song song công việc

Nhóm A - Sinh viên IT năm 2-4

Bối cảnh và chiến lược

Bạn có thứ vũ khí mạnh nhất: thời gian. Hãy dùng nó để xây nền tảng thật vững - đây là cơ hội mà người đi làm rồi sẽ ghen tị. Mục tiêu sau 12 tháng: có 1 portfolio đủ để xin internship hoặc Junior DevOps tại các công ty IT mid-size trở lên.

Phân bổ thời gian khuyến nghị

- **Trong kỳ học:** 10-12 giờ/tuần - 1.5 giờ/ngày + cuối tuần 4 giờ.
- **Hè và Tết:** dồn 25-30 giờ/tuần - đây là giai đoạn bứt phá nhất.
- **Tổng cộng:** ~700-800 giờ trong 12 tháng. Đủ để hoàn thành 10 bước nền tảng + bắt đầu 2 năng lực lớp AI 2026.

Plan theo tháng

Tháng	Trọng tâm	Sản phẩm cuối tháng
1-2	Linux + Networking	1 home lab 2 VM, 1 shell script automation, blog post 1 bài
3	Git + Scripting cơ bản	Repo Python CLI tool, PR đầu tiên vào open-source
4-5	CI/CD + Cloud (AWS)	AWS account active, 1 web app deploy thành công lên cloud
6-7	Container + Kubernetes	App được container hoá, chạy trên local K8s, có Helm chart
8	Terraform	Toàn bộ AWS infrastructure được IaC bằng Terraform
9	Monitoring	Prometheus + Grafana setup, 3 alert rule có ý nghĩa
10	DevSecOps cơ bản	Pipeline có 4 security gate
11	Chọn 1 năng lực AI 2026 đào sâu	Dự án portfolio chính: AIOps / IDP / FinOps
12	CV, LinkedIn, interview prep	CV chỉn chu + GitHub có 5+ project chất lượng + LinkedIn active

Hành động cụ thể trong 30 ngày đầu

- Tạo tài khoản GitHub, viết bio rõ ràng, dùng tên thật (không nickname)
- Mở blog kỹ thuật (Hashnode, dev.to, hoặc Substack - free)
- Tham gia 1-2 community: Vietnam DevOps Telegram/Discord, r/devops trên Reddit
- Đặt mục tiêu commit ≥ 4 ngày/tuần vào GitHub trong 30 ngày đầu - kỷ luật là kỹ năng quan trọng nhất
- Đọc xong 'The Linux Command Line' chương 1-10
- Hoàn thành mini-project bước 1 (Linux), push lên GitHub có README đầy đủ

Nhóm B - Fresher / Junior Developer chuyển hướng

Bối cảnh và chiến lược

Bạn đã có lợi thế lớn: hiểu code, biết Git, đã trải nghiệm phần mềm thật. Phần thiếu chủ yếu là infrastructure và vận hành. Tin tốt: ba bước đầu (Linux, Networking, Git) có thể đi nhanh. Tin xấu: nhiều người chủ quan, bỏ qua nền tảng quá nhanh và mãi không lên được. Đừng phạm lỗi này.

Chiến lược 6-9 tháng tập trung

- **Giai đoạn 1 - Lấp lỗ hổng (Tháng 1-2):** Linux nâng cao, Networking, Bash scripting. Đừng vội nhảy sang Docker.
- **Giai đoạn 2 - Tăng tốc delivery (Tháng 3-5):** CI/CD, Cloud (AWS), Container, IaC. Đây là phần lợi thế của bạn - code đã có, infra đang học.
- **Giai đoạn 3 - Bứt phá (Tháng 6-9):** Monitoring, Security, và CHỌN 1 năng lực lớp 2026 để làm sâu thay vì chạy theo cả 5.

Sai lầm phổ biến của nhóm này

- Học Docker, K8s, Helm cùng lúc trước khi nắm Linux - kết quả: dùng được nhưng không debug được.
- Chỉ làm tutorial, không có dự án end-to-end của riêng mình - recruiter không phân biệt được với người khác.
- Tránh học Networking vì 'lý thuyết' - đúng đến khi production có vấn đề và bạn không biết bắt đầu từ đâu.
- Học cả 5 năng lực lớp 2026 cùng lúc - kết quả: không sâu cái nào, không có dự án ấn tượng.

Tận dụng lợi thế code

Phần này khác biệt rõ với Nhóm A: bạn có thể (và nên) viết tool/script tự động hoá DevOps task bằng Python ngay từ đầu. Mọi mini-project nên tích hợp 1 yếu tố code mạnh hơn (REST API, CLI tool, library). Đây là phần Nhóm A sẽ chậm hơn - bạn dùng nó làm lợi thế cạnh tranh.

Nhóm C - IT chuyển hướng (Ops / QA / Tester)

Bối cảnh và chiến lược

Bạn có thứ Nhóm A và B đều thiếu: kinh nghiệm vận hành thật. Bạn đã từng thấy production cháy, đã debug network ở 2 giờ sáng, đã viết test cho service phức tạp. Đây là vốn quý, đừng coi nhẹ. Phần cần bổ sung: kỹ năng code, tư duy tự động hoá, và quy trình CI/CD hiện đại.

Lộ trình rút gọn 6-9 tháng song song công việc

Tháng	Trọng tâm	Sản phẩm
1-2	Scripting (Python ưu tiên) + Git nâng cao	Repo cá nhân với 2-3 Python tool tự động hoá task hiện tại của công việc
3	CI/CD (GitHub Actions)	Pipeline đầy đủ cho repo Python ở tháng 1-2
4-5	Cloud (AWS) + Container	Migrate 1 app nội bộ lên AWS có container hoá
6	IaC (Terraform) + Monitoring	Toàn bộ infrastructure được IaC, có monitoring stack
7-9	DevSecOps + chọn 1 năng lực lớp 2026	Dự án bứt phá: AIOps / IDP / FinOps

Tận dụng vai trò hiện tại

Đây là chiến lược then chốt: tự động hoá CHÍNH công việc hằng ngày của bạn bằng skill mới học. Mỗi bước trong lộ trình đều có thể áp dụng vào task hiện tại - đó là cách học hiệu quả gấp 3 lần đọc sách.

- Ops: viết script tự động hoá 5 task đang phải làm tay → portfolio + giá trị cho công ty
- QA/Tester: build CI/CD pipeline có integration test, đề xuất chuyển từ test thủ công sang test automation
- DBA: viết Terraform module quản lý RDS, schedule backup tự động, monitoring với Prometheus exporter

> **LỢI THẾ NGẦM CỦA NHÓM C** *Recruiter thường ưu tiên người có background vận hành thật vì rủi ro thấp hơn. Bạn không cần học code giỏi bằng Junior Dev - bạn cần biết đủ để tự động hoá phần việc đang làm.*

> *Đề xuất internal transfer (chuyển nội bộ) - đây là path êm ái nhất, không cần đổi công ty.*

CHƯƠNG 5 - Lộ trình 12 tháng (bản chung)

Đây là lộ trình tổng hợp 3 nhóm thành 1 timeline chung - dùng làm bản đồ định hướng. Chia thành 3 pha lớn, mỗi pha có mục tiêu và deliverable rõ ràng.

Pha 1 - Nền tảng vững (Tháng 1-3)

Đây là pha quan trọng nhất. Nếu pha này không vững, toàn bộ phía sau lung lay. Đừng cố tăng tốc trong giai đoạn này - đầu tư đúng chỗ sẽ trả về gấp 10 lần ở các pha sau.

Mục tiêu pha 1

- Nắm vững Linux, Networking, Git, Scripting - đây là 4 trụ không thể thiếu
- Quen với việc commit code lên GitHub mỗi ngày
- Có blog kỹ thuật và post được 2-3 bài đầu tiên
- Tham gia ít nhất 1 community DevOps Việt Nam

Deliverable cuối pha 1

- 3 mini-project commit lên GitHub: shell automation script, Python CLI tool, home lab setup
- Tài khoản AWS Free Tier active (chưa cần làm gì lớn, chỉ để quen UI)
- Hiểu rõ và phân biệt được 50+ thuật ngữ cơ bản (xem Phụ lục B - Glossary)

Pha 2 - Thực chiến delivery (Tháng 4-7)

Pha này tập trung vào kỹ năng delivery: làm sao đưa code từ máy lên production một cách tự động, an toàn, lặp lại được. Đây là phần phân biệt người 'biết DevOps lý thuyết' với người 'làm được DevOps'.

Mục tiêu pha 2

- Build được pipeline CI/CD end-to-end cho ít nhất 1 ứng dụng thật
- Deploy được app lên AWS với 3-tier architecture đúng chuẩn
- Container hoá và điều phối được app trên Kubernetes
- Quản lý hạ tầng bằng Terraform - không bao giờ click console

Deliverable cuối pha 2

- 1 web app production-ready: code → GitHub → CI/CD → Docker image → K8s → AWS
- Terraform module reuse được, có state remote, có CI gate cho IaC
- Tài liệu architecture diagram chỉn chu trong README - recruiter sẽ xem cái này
- Đăng được 5-7 bài blog kỹ thuật về quá trình học

Pha 3 - Chuyên sâu & bứt phá (Tháng 8-12)

Pha này là phần phân biệt người 'sẵn sàng đi làm Junior' với người 'sẵn sàng đi làm Mid-level hoặc cao hơn'. Đây là lúc chọn hướng đào sâu và xây bản sắc kỹ thuật cá nhân.

Mục tiêu pha 3

- Bổ sung Monitoring & Observability - kỹ năng phân biệt chín và sống
- Tích hợp Security-as-Code vào toàn bộ pipeline
- Chọn 1-2 năng lực lớp AI 2026 để đào sâu - đây là điểm khác biệt cạnh tranh
- Hoàn thiện CV, LinkedIn, portfolio cho việc đi làm hoặc nâng cấp vị trí

Deliverable cuối pha 3 - sẵn sàng đi làm

- Project bứt phá: ít nhất 1 dự án thể hiện lớp AI 2026 (AIOps / IDP / MCP agent / FinOps / Security-as-Code)
- CV 1 trang chỉn chu, có 5+ project nổi bật trên GitHub
- LinkedIn profile active, đã viết 10-15 bài blog kỹ thuật
- Pass được vòng phỏng vấn technical đầu tiên với 1 công ty thật

Bảng tổng hợp 12 tuần đầu - hành động cụ thể

Để bạn không bị 'tê liệt vì quá nhiều việc', đây là bảng cụ thể từng tuần cho 12 tuần đầu - giai đoạn quan trọng nhất.

Tuần	Topic	Action cụ thể
1	Setup môi trường	Cài Ubuntu/WSL2, tạo GitHub, mở blog, join community
2	Linux căn bản	Học 30 lệnh CLI thiết yếu, viết script đầu tiên (file backup)
3	Linux nâng cao	Systemd, journal, permissions, viết unit file
4	Bash scripting	Script monitor disk + alert webhook, đẩy GitHub
5	Networking - OSI/TCP	Vẽ sơ đồ HTTPS request flow, dùng curl debug
6	Networking - DNS/TLS	Setup local DNS server, capture TLS handshake
7	Git fundamentals	Branching, merge vs rebase, conflict resolution
8	Git advanced	Mở 1 PR vào open-source project (typo cũng được)
9	Python cơ bản	Hoàn thành chương 1-8 sách Automate the Boring Stuff
10	Python CLI tool	Build CLI gọi GitHub API, xuất report markdown
11	CI cơ bản	Setup GitHub Actions cho repo Python: lint + test
12	CI nâng cao + Cloud account	Pipeline đa stage có cache; mở AWS account, hoàn thành lab IAM cơ bản

CHƯƠNG 6 - Bốn kỹ năng AI không thay được

Công cụ sẽ đổi mỗi năm. Bốn thứ dưới đây thì không. Đầu tư vào chúng là khoản đầu tư không bao giờ lỗi thời. Đây cũng là phần phân biệt người chuyên gia thật sự với người 'biết dùng tool'.

1. Tư duy hệ thống

Nhìn vấn đề như một mạng lưới các thành phần tương tác, không phải các phần riêng lẻ. Hiểu rằng tối ưu một bộ phận có thể làm xấu tổng thể.

Cách rèn: Luyện bằng cách: mỗi sự cố production, vẽ sơ đồ và truy nguyên nhân ngược dòng tới gốc rễ. Đọc 'Thinking in Systems' của Donella Meadows. Tham gia incident review của team - không phải chỉ nghe, mà ghi chép và tự rút bài học.

2. Phán đoán kiến trúc và đánh đổi

Mọi quyết định kỹ thuật là sự đánh đổi (trade-off): tốc độ vs chi phí, đơn giản vs linh hoạt, an toàn vs trải nghiệm. AI có thể đề xuất, nhưng quyết định cuối là việc của con người vì AI không nắm được bối cảnh tổ chức.

Cách rèn: Mỗi lần thiết kế hoặc đề xuất, viết ra 3 phương án, liệt kê trade-off của từng cái. Đây gọi là Architecture Decision Record (ADR). Tham khảo: thoughtworks.com/insights/blog/architecture/decision-records-record-decision.

3. Thiết kế guardrail và quản trị rủi ro

Đây là phần việc giá trị nhất 2026: định nghĩa luật chơi cho AI và automation vận hành an toàn. Tốc độ AI sinh code → ai biết đặt guardrail tốt nhất sẽ thắng.

Cách rèn: Học OPA / Policy-as-Code. Đọc cách Google SRE thiết kế error budget. Hỏi mỗi lần triển khai: nếu cái này hỏng vào 2 giờ sáng, ai phát hiện ra, ai sửa, làm sao rollback an toàn?

4. Giao tiếp và ra quyết định

Làm việc với con người, thuyết phục, chịu trách nhiệm cuối cùng. Đây là vùng AI không chạm tới và cũng là vùng phân biệt Engineer và Senior Engineer rõ nhất.

Cách rèn: Viết technical document mỗi tuần - cho team, cho cộng đồng, cho blog. Trình bày trong team meeting ít nhất 1 lần mỗi tháng. Đọc 'The Staff Engineer's Path' của Tanya Reilly khi đã có 2-3 năm kinh nghiệm.

> **NGUYÊN TẮC ĐẦU TƯ CHO 4 KỸ NĂNG NÀY** *Không có khoá học chuyên biệt. Cách duy nhất là LÀM - kèm phản tỉnh.*

> *Mỗi cuối tuần dành 30 phút viết nhật ký kỹ thuật: tuần này gặp vấn đề gì, quyết định thế nào, nhìn lại có gì cần thay đổi?*

> *Tìm 1 mentor cấp Senior - gặp 1 lần/tháng, hỏi cụ thể, đừng hỏi chung chung 'em nên học gì'.*

CHƯƠNG 7 - Hành động 14 ngày đầu tiên

Cuốn tài liệu này dài. Đừng để bị tê liệt. Đây là checklist cụ thể cho 14 ngày đầu - làm xong các bước này, bạn đã chính thức trên đường.

Ngày 1 · Setup môi trường - Cài Ubuntu 22.04 (hoặc WSL2 nếu Windows). Cài VS Code. Cài tmux. Cài Git.

Ngày 2 · GitHub & Identity - Tạo GitHub account tên thật. Setup SSH key. Viết bio. Tạo repo learn-devops với README giới thiệu lộ trình của bạn.

Ngày 3 · Blog - Mở blog trên Hashnode hoặc dev.to. Viết bài đầu tiên: 'Tôi bắt đầu lộ trình DevOps' - chia sẻ mục tiêu, thời gian dự kiến.

Ngày 4 · Linux căn bản - Học 20 lệnh CLI: ls, cd, cp, mv, rm, cat, less, grep, find, ps, top, df, du, chmod, chown, sudo, ssh, tar, curl, wget. Tự thử mọi flag.

Ngày 5 · Linux căn bản (tiếp) - Học systemd. Tạo 1 service unit file. Enable, start, status, logs với journalctl.

Ngày 6 · Linux - File system - Đọc man hier. Vẽ sơ đồ /etc, /var, /usr, /home - viết blog post giải thích.

Ngày 7 · Tổng kết tuần 1 - Review checklist. Cập nhật file Skills_Checklist_2026.xlsx. Đăng 1 cập nhật ngắn trên LinkedIn về tuần học đầu tiên.

Ngày 8 · Bash scripting - Viết script đầu tiên: backup folder /etc lên /tmp/backup-YYYYMMDD.tar.gz. Test thử.

Ngày 9 · Bash scripting (tiếp) - Mở rộng script: thêm error handling (set -euo pipefail), logging vào file, cleanup file cũ > 7 ngày.

Ngày 10 · Networking căn bản - Học OSI 7 layer. Dùng dig, curl -v để xem 1 HTTP request thật. Viết blog 'OSI cho người mới - bằng ví dụ thật'.

Ngày 11 · Networking - DNS - Tìm hiểu A, CNAME, MX, TXT record. Dùng dig kiểm tra DNS của domain bất kỳ. Viết note tự học.

Ngày 12 · Git căn bản - Đọc chương 1-3 sách Pro Git. Tạo repo cá nhân, thử commit, branch, merge, conflict resolution.

Ngày 13 · Git workflow - Học GitHub Flow. Thử mở 1 PR trong repo của chính mình (từ branch khác). Review PR như đang review code người khác.

Ngày 14 · Mini-project & tổng kết - Hoàn thành mini-project bước 1 (Linux automation script), đẩy lên GitHub có README chỉn chu. Update checklist.

> **SAU 14 NGÀY** *Bạn đã có 5-7 commit trên GitHub, 2-3 bài blog, hiểu Linux ở mức căn bản, biết Git workflow.*

> *Đây chính là dấu hiệu BẠN ĐÃ BẮT ĐẦU. Đa số bỏ cuộc trong 14 ngày đầu vì không thấy tiến bộ - bạn vừa qua được phần khó nhất.*

> *Tiếp tục: mỗi tuần review checklist, mỗi tháng nhìn lại 1 lần. Chỉ cần kỷ luật, đích đến đã được vẽ sẵn.*

CHƯƠNG 8 - Bảy quy tắc vận hành cá nhân

Đây là quy tắc em đã rút ra sau 8 năm làm nghề và đào tạo. Không phải lý thuyết - đây là những thứ em ước có người nói với em từ năm 2018.

1. Học để LÀM, không học để BIẾT

Khi học xong 1 khái niệm, dừng video, mở terminal, làm thử. Nếu không có dự án áp dụng được, kiến thức đó tan biến trong 2 tuần. Quy tắc 50/50: 50% thời gian học lý thuyết, 50% thời gian áp dụng - không hơn không kém.

2. Một công cụ, học sâu trước

Đừng học Terraform, Pulumi, Crossplane cùng lúc. Học sâu Terraform trước. Cái khác là biến thể của cùng tư duy. Sau khi master 1 tool, học tool thứ 2 chỉ mất 1/3 thời gian. Người mới hay sa đà vào việc 'thử mọi thứ' và mãi không sâu cái gì.

3. GitHub là CV của bạn

Recruiter check GitHub trước CV. Project chỉn chu có README đầy đủ giá trị bằng 5 dòng trong CV. Quy tắc: mỗi mini-project phải có README giải thích vấn đề, giải pháp, cách dùng, architecture diagram nếu cần. Tránh repo 'tutorial-1', 'tutorial-2' không context.

4. Khi tilt, ngừng học

Khi cảm thấy quá tải, frustrated, không hiểu gì cả - đó là dấu hiệu cần dừng lại. Có thể bạn nhảy bước, hoặc tâm trí cần nghỉ. Một ngày off có thể tiết kiệm 1 tuần học sai hướng. Đừng nhồi nhét.

5. Cộng đồng quan trọng hơn khoá học

Tham gia 1-2 community DevOps Việt Nam (Telegram, Discord, Slack). Hỏi câu cụ thể, đừng hỏi 'em nên học gì'. Trả lời 1 câu hỏi/tuần của người khác - dạy lại là cách học nhanh nhất.

6. Viết - public - là kỹ năng then chốt

Mỗi 2 tuần, viết 1 bài blog kỹ thuật. Không cần dài, không cần hoàn hảo. Viết để rèn tư duy có cấu trúc và để tạo dấu chân kỹ thuật trên internet. Recruiter Google tên bạn - bài blog của bạn xuất hiện = bạn vừa được tuyển.

7. Tốc độ không phải mục tiêu - kiên trì mới là

Người học DevOps trong 6 tháng rồi bỏ - thua người học 18 tháng đều đặn. Đặt nhịp 4 ngày commit/tuần, 1 bài blog/2 tuần, 1 mini-project/tháng. Duy trì 12 tháng, kết quả tự tới.

Phụ lục A - Resource list

Tổng hợp tài nguyên đã được verified - em đã đọc/dùng và đảm bảo chất lượng. Không phải affiliate link, không phải sponsored.

Sách thiết yếu (đọc đến cuối lộ trình)

- "The Phoenix Project" - Gene Kim et al. - Tiểu thuyết DevOps. Đọc trong tuần 1 để hiểu bối cảnh và động lực.
- "The Linux Command Line" - William Shotts - Bible cho Linux căn bản. Free PDF.
- "Pro Git" - Scott Chacon - Bible cho Git. Free online.
- "Continuous Delivery" - Jez Humble & David Farley - Sách gối đầu cho CI/CD.
- "Kubernetes Up & Running" - Brendan Burns et al. - Vào K8s ở mức operational, không phải developer.
- "Site Reliability Engineering" - Google - Free online. Đọc chương về SLO, error budget, postmortem.
- "Terraform: Up & Running" - Yevgeniy Brikman - Sách Terraform chuẩn nhất hiện tại.
- "Designing Data-Intensive Applications" - Martin Kleppmann - Khi đã có 1-2 năm kinh nghiệm, đọc cuốn này.

Khoá học online (chọn vài cái phù hợp)

- Stephane Maarek - AWS Certified Solutions Architect Associate (Udemy, ~$15 khi giảm)
- Nigel Poulton - Getting Started with Kubernetes (Pluralsight)
- KodeKloud - DevOps Engineer course (rẻ, lab nhiều)
- HashiCorp Learn - Terraform tutorials (free, official)
- CS50P - Harvard Introduction to Programming with Python (free, edX)
- AWS Skill Builder - phần free tier có nhiều lab chất lượng
- Linux Foundation - LFS158 Introduction to Kubernetes (free)

Blog & YouTube

- Cloudflare Learning Center - chất lượng best in class về networking
- Cilium blog - deep technical về cloud-native networking
- Honeycomb blog - observability từ team xây tooling
- Netflix Tech Blog - case study real production scale
- NetworkChuck (YouTube) - networking dễ hiểu cho người mới
- TechWorld with Nana (YouTube) - DevOps overview tốt
- DevOps Việt Nam (Facebook Group, Telegram) - community VN active

Tools nên cài sẵn trên máy

- Editor: VS Code + extension Remote-SSH, Docker, Kubernetes, HashiCorp Terraform
- Terminal: Windows Terminal (Win) hoặc iTerm2 (Mac), kèm zsh + oh-my-zsh
- Container: Docker Desktop hoặc Podman + nerdctl
- Kubernetes local: kind hoặc minikube
- Cloud CLI: aws-cli v2, kubectl, helm, terraform, gcloud nếu cần
- Productivity: tmux, fzf, ripgrep (rg), bat, jq, yq, httpie hoặc httpx

Phụ lục B - Glossary 30 thuật ngữ cốt lõi

Định nghĩa ngắn để bạn không phải Google mỗi lần gặp thuật ngữ. Học dần qua thực hành - không cần thuộc lòng.

Thuật ngữ	Định nghĩa
AIOps	Việc áp dụng AI/ML vào vận hành IT - phát hiện anomaly, phân loại alert, tự xử lý.
Artifact	Sản phẩm trung gian/cuối của pipeline build: file ZIP, container image, binary, npm package.
Blue-Green Deployment	Triển khai có 2 môi trường giống hệt (blue đang chạy, green stand-by). Đẩy version mới lên green, switch traffic. Rollback nhanh.
Canary Deployment	Triển khai version mới cho 1 phần nhỏ người dùng (5-10%), tăng dần nếu OK.
CI / CD / CDe	Continuous Integration (tích hợp liên tục) / Continuous Delivery (sẵn sàng deploy) / Continuous Deployment (tự deploy production).
Container	Đóng gói app + dependencies vào 1 đơn vị chạy được isolated. Dùng namespace + cgroup của Linux.
Control Plane	Phần điều khiển của hệ thống (vs data plane - phần xử lý dữ liệu). K8s control plane gồm API server, scheduler, controller manager, etcd.
DevSecOps	Tích hợp security vào DevOps workflow - shift-left security, không phải dán cuối.
Error Budget	Tỉ lệ downtime cho phép trong 1 chu kỳ. Vd SLO 99.9% → budget = 43 phút/tháng.
FinOps	Quản lý chi phí cloud một cách có hệ thống - cross-function giữa Engineering, Finance, Business.
GitOps	Dùng Git làm nguồn sự thật cho hạ tầng và deployment. Mọi thay đổi qua PR. ArgoCD/Flux là tool phổ biến.
Golden Path	Con đường khuyến nghị mà developer đi theo - có sẵn template, security, observability tích hợp.
Helm Chart	Package manager cho Kubernetes. Chart = template + values, render ra manifest.
IaC (Infrastructure as Code)	Khai báo hạ tầng bằng code, version control được, repeatable. Terraform là chuẩn de-facto.
IDP (Internal Developer Platform)	Platform nội bộ self-service cho developer - không cần mở ticket xin môi trường.
Ingress	Object Kubernetes định nghĩa quy tắc route HTTP/HTTPS từ ngoài vào Service trong cluster.
IAM (Identity and Access Management)	Hệ thống quản lý ai (user/service) có quyền gì với tài nguyên gì.
MCP (Model Context Protocol)	Chuẩn kết nối AI agent với hệ thống thật (file, API, database) - Anthropic giới thiệu 2024.

Thuật ngữ	Định nghĩa
Observability	Khả năng hiểu trạng thái nội bộ hệ thống chỉ qua output bên ngoài. 3 trụ: metrics, logs, traces.
OPA (Open Policy Agent)	Engine policy-as-code - viết quy tắc bằng Rego, áp dụng vào K8s, Terraform, API, etc.
Pod	Đơn vị triển khai nhỏ nhất trong K8s. Chứa 1+ container chia sẻ network và storage.
RBAC (Role-Based Access Control)	Cấp quyền dựa trên vai trò, không phải user trực tiếp. Tiêu chuẩn của K8s và cloud.
SRE (Site Reliability Engineer)	Vai trò tập trung vào độ tin cậy hệ thống. Google sáng tạo ra khái niệm này.
SLO / SLA / SLI	Service Level Objective (mục tiêu nội bộ) / Agreement (cam kết khách hàng) / Indicator (chỉ số đo).
State (Terraform)	File ghi trạng thái hạ tầng Terraform đang quản lý. Lưu remote (S3 + lock) trong production.
Service Mesh	Hạ tầng quản lý giao tiếp service-to-service: routing, retry, mTLS, observability. Vd Istio, Linkerd.
Service Account	Identity cho non-human (app, pipeline) khi gọi API. Khác user thường.
TLS / mTLS	Transport Layer Security (HTTPS) / mutual TLS (cả 2 phía verify cert lẫn nhau).
Trace / Span	Trace = hành trình 1 request qua nhiều service. Span = 1 đoạn (1 service) trong trace.
Vibe Coding	Lập trình bằng cách mô tả ý định cho AI - code do AI sinh ra. Mới phổ biến 2024-2026, kèm rủi ro nếu không có guardrail.

Phụ lục C - Liên hệ và bước tiếp theo

Đi xa hơn với cộng đồng

Tài liệu này là điểm bắt đầu. Để duy trì động lực và tránh học một mình, em khuyến khích bạn kết nối:

- **Theo dõi kênh TikTok / YouTube:** @codersapthatnghiep - nội dung kỹ thuật ngắn về DevOps, AI, automation.

Khi bạn cần đi nhanh hơn

Tự học hoàn toàn được. Em đã từng tự học. Nhưng nó có thể mất 12-18 tháng và rất dễ lan man - vì không có ai phản biện cách làm của bạn.

Nếu bạn muốn rút ngắn lộ trình và có người đồng hành, em đang vận hành chương trình đào tạo DevOps thực chiến với:

- Lộ trình theo 1-1 mentoring, review code và project portfolio
- Học dự án thật từ production setup - không phải lab giả
- Community kín cho học viên, được kết nối với cựu học viên đã đi làm tại các công ty mid-large
- Cam kết kết quả rõ ràng - sẵn sàng đi làm Junior DevOps sau 4-6 tháng tập trung

> **LIÊN HỆ TƯ VẤN LỘ TRÌNH** *Email: congdinh2021@gmail.com*
>
> *Zalo / Telegram: 0944551356*
>
> *Hoặc đặt 1 buổi tư vấn miễn phí 30 phút qua: congdinh2021@gmail.com*

Chúc bạn 12 tháng tới đầy năng lượng. Hẹn gặp ở phía bên kia.

- Đinh Xuân Công